Universidad Tecnológica de Santiago (UTESA)



Nombre:

Erick Marte

Materia:

Algoritmo Paralelos

Profesor:

Ivan Mendoza

Matrículas:

1-21-2841

Asignación:

Tarea 5

1. Objetivo de la práctica

Evaluar cómo responde una arquitectura serverless de AWS cuando varias personas hacen solicitudes al mismo tiempo. Para ello se realizaron pruebas con Grafana k6 al endpoint POST /notifications/send, observando los tiempos de respuesta, los errores y el comportamiento de los servicios que intervienen en el envío de notificaciones.

RESULTADO GENERAL

Con 10 usuarios virtuales, las 30 solicitudes fueron aceptadas. Al lanzar una ráfaga de 50 usuarios, únicamente 10 fueron aceptadas y 40 fallaron. El monitoreo de AWS mostró que la cuota regional disponible era de 10 ejecuciones simultáneas.

2. Sistema evaluado

La aplicación envía una petición HTTP a API Gateway. Este servicio pasa la solicitud a la Lambda principal, que ejecuta el backend de Spring Boot. Si se acepta el mensaje, se publica en SNS y se entrega a una cola SQS. Una segunda Lambda, escrita en Python, toma los mensajes de la cola y solicita el envío mediante Amazon SES.

Aplicación móvil → API Gateway → Lambda Java/Spring Boot
↓
SNS → SQS → Lambda Python → SES Simulator

Se utilizó el simulador oficial de SES para no llenar un correo personal de mensajes. Esto permite comprobar el procesamiento de las solicitudes, aunque no demuestra entrega real a Gmail. Las pruebas siguen utilizando recursos de AWS.

3. Herramientas y forma de trabajo

- Grafana k6: simuló los usuarios virtuales y registró las respuestas HTTP.
- AWS CloudWatch: permitió revisar invocaciones, tiempos, concurrencia, limitaciones y registros de las funciones Lambda.
- AWS SNS y SQS: mostraron la publicación y el movimiento de los mensajes por la arquitectura.
- Visual Studio Code y Codex: se usaron para preparar los scripts, recopilar las medidas y organizar el informe.

4. Configuración de las pruebas

Prueba	Usuarios virtuales	Duración configurada	Solicitudes máximas
Actividad 1 — línea base	10	1 minuto	30 (3 por usuario)
Actividad 2 — primer escenario	50	2 minutos	50 (1 por usuario)

Las pruebas tuvieron un límite de solicitudes para controlar el consumo. La de 50 usuarios envió una ráfaga inicial y después mantuvo a los usuarios esperando; por tanto, ninguna de estas dos pruebas representa tráfico sostenido durante todo el tiempo configurado.

5. Resultados obtenidos

5.1. Solicitudes y errores

Prueba	Solicitudes	Aceptadas (202)	Fallidas	Errores
10 usuarios	30	30	0	0 %
50 usuarios	50	10	40	80 %

Resultados reales de los dos ensayos controlados

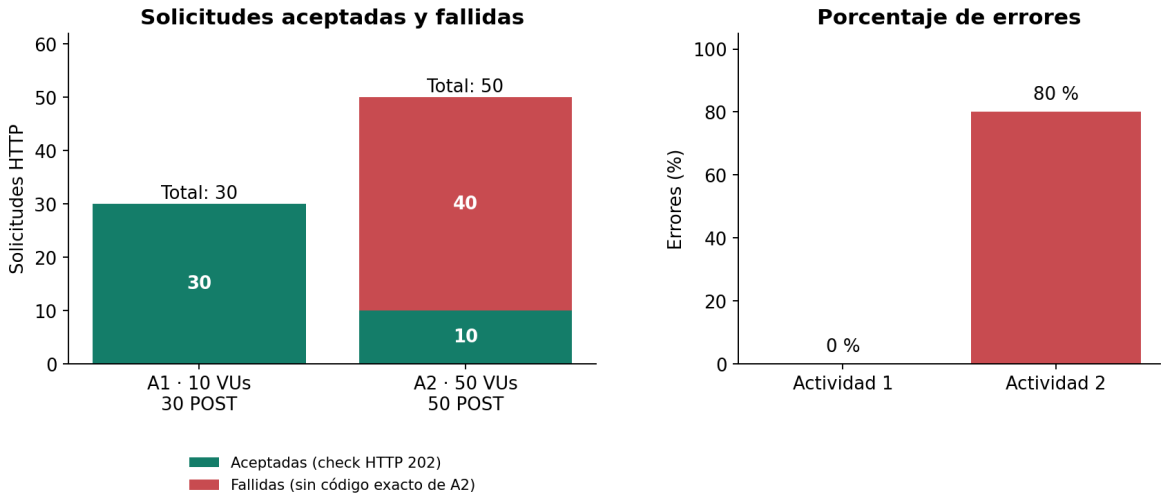
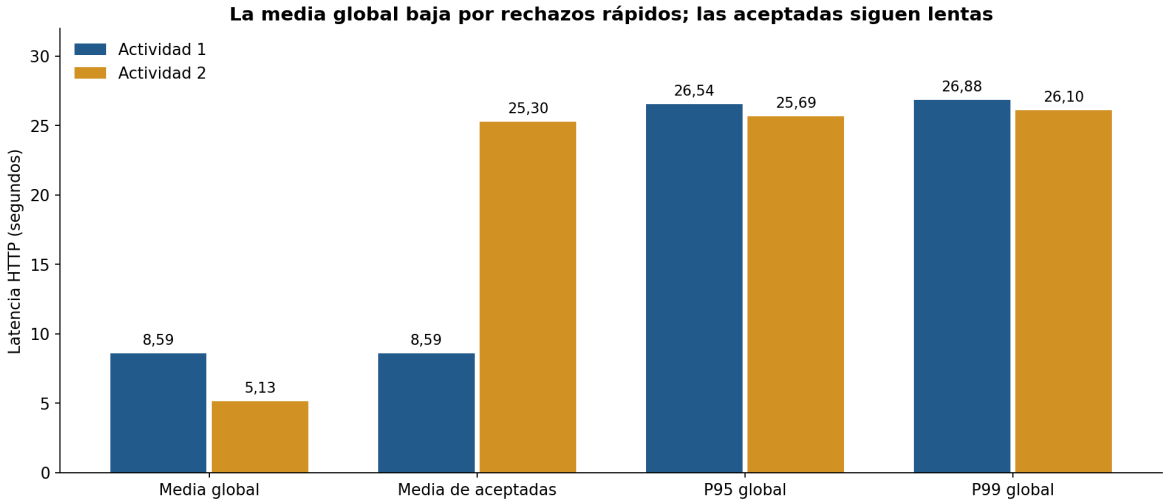


Figura 1. Solicitudes aceptadas y fallidas en las dos pruebas controladas. Fuente: resumen k6 aportado.

5.2. Tiempos de respuesta

Indicador	10 usuarios	50 usuarios
Promedio global	8,59 s	5,13 s
Mínimo	0,095 s	0,075 s
Máximo	26,89 s	26,46 s
P95	26,54 s	25,69 s
P99	26,88 s	26,10 s
Promedio de respuestas aceptadas	8,59 s	25,30 s

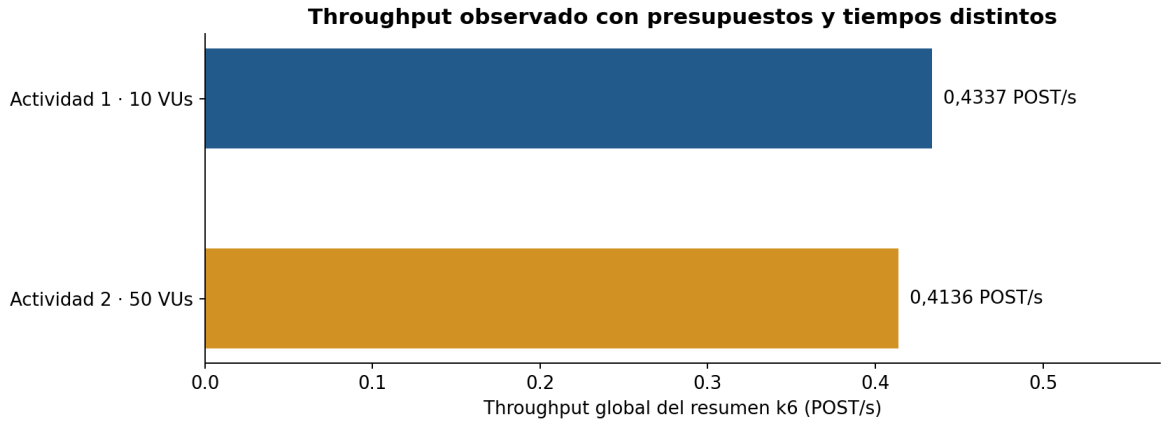
El promedio general parece más bajo con 50 usuarios, pero eso no significa que el sistema mejoró: las 40 respuestas fallidas fueron rápidas. Al separar las 10 solicitudes aceptadas, su tiempo promedio fue de unos 25,30 segundos. La media estimada del conjunto de las 40 fallidas fue de 86,09 milisegundos, calculada a partir de los promedios; no son tiempos individuales reconstruidos.



Agregados originales: no reconstruyen una distribución ni una serie temporal. Máximo 50 muestras.

5.3. Solicitudes por segundo

En el resumen global, la Actividad 1 registró aproximadamente 0,4337 POST/s y la Actividad 2, 0,4136 POST/s. Estas cifras se calcularon sobre el tiempo total del ensayo, incluyendo esperas y fallos; no deben interpretarse como el rendimiento máximo del backend.



Incluye espera del escenario y margen final. No equivale a capacidad máxima ni tráfico sostenido.

6. Monitoreo de AWS y cuello de botella

6.1. Lambda principal

Al analizar la prueba de 50 usuarios en CloudWatch se observó un máximo de 10 ejecuciones simultáneas y un pico de 40 limitaciones o throttles. La página de cuotas de AWS confirmó que la cuenta tenía un máximo regional aplicado de 10 ejecuciones. Esta coincidencia respalda que la capacidad concurrente disponible fue el cuello de botella operativo de esa ráfaga.



En el intervalo mostrado en CloudWatch, las invocaciones de la Lambda principal sumaban 47. Esa cifra corresponde al intervalo completo seleccionado, no únicamente a las 50 solicitudes del ensayo.

6.2. Arranques lentos de Spring Boot

También se observaron registros de inicialización de Lambda que terminaron por timeout a los 10 segundos. Algunas solicitudes aceptadas tardaron cerca de 25 segundos, lo que es compatible con arranques en frío prolongados. El inicio de Spring Boot, Hibernate/JPA y conexiones de base de datos requiere un análisis más detallado para atribuir el tiempo exacto a cada componente.

6.3. SNS, SQS y Lambda consumidora

En SNS se observó un pico de 30 publicaciones y 30 entregas a la suscripción de SQS durante la línea base. SQS mostró recepción y eliminación de mensajes, y las colas principal y de fallidos aparecían sin mensajes pendientes cuando se consultaron. Estos gráficos agrupados no prueban por sí solos cuántos mensajes únicos completaron todas las etapas.

La Lambda de notificaciones registró 31 invocaciones en el intervalo de tres horas mostrado, un tiempo promedio de 267 ms, cero errores de ejecución y un pico de una limitación. Los registros de CloudWatch incluyeron mensajes procesados de SQS con identificadores devueltos por SES. Eso respalda la aceptación de envíos simulados, no la entrega a un buzón real.

CÓMO INTERPRETAR HTTP 202

Una respuesta 202 indica que la API aceptó la solicitud y la pasó al flujo asíncrono. No significa que todo el envío de correo haya terminado en ese mismo momento.

7. Actividad 4 — Análisis de estrés y degradación

La ráfaga controlada de 50 usuarios mostró una degradación clara frente a la primera prueba: el porcentaje de errores pasó de 0 % a 80 %. El sistema no pudo aceptar las 50 solicitudes simultáneas bajo la cuota regional disponible.

Esto permite señalar un límite operativo de la configuración observada, pero no equivale a una prueba de estrés incremental completa. No se ejecutaron incrementos progresivos hasta 750, 1.000 o 2.000 usuarios, ni se obtuvo una curva que permita ubicar el punto de ruptura general de la aplicación.

8. Escenarios de 100, 250 y 500 usuarios

Escenario	Duración prevista	Estado	Resultados
100 usuarios	2 minutos	Preparado, bloqueado	No medidos

250 usuarios	2 minutos	Preparado, bloqueado	No medidos
500 usuarios	2 minutos	Preparado, bloqueado	No medidos

Los scripts están preparados para documentación, pero quedaron bloqueados antes de enviar HTTP. Se tomó esta decisión para conservar la cuota de la cuenta y evitar solicitar un aumento a 1.000 ejecuciones sin necesidad. No se incluyen resultados inventados para estas pruebas.

9. Actividad 5 — Recuperación

Después de una carga es importante comprobar cuánto tarda el sistema en estabilizarse. Para la cola, la recuperación sería volver al nivel habitual de mensajes pendientes y terminar de procesar los aceptados. Para la API, sería necesario observar solicitudes posteriores atendidas correctamente.

En este caso no se dispone de una serie histórica completa con las horas precisas antes, durante y después de la ráfaga. Por tanto, no es correcto afirmar que el servicio se recuperó en un número determinado de segundos. Las capturas con colas vacías son un indicio puntual, no una medición de recuperación.

9.1. Cómo medirla con los registros existentes

- Ubicar la hora del último POST de la ráfaga mediante registros reales.
- Revisar en SQS mensajes visibles, mensajes en tránsito, edad del más antiguo y cola de fallidos.
- Comprobar en Lambda invocaciones, throttles, errores, duración y registros de la consumidora.
- Determinar la hora de estabilización con varios puntos consecutivos de la serie histórica y calcular la diferencia de tiempos.
- Separar la recuperación de SQS de una nueva respuesta HTTP exitosa; una cola vacía no confirma por sí sola que la API haya recuperado su capacidad.

MEDICIÓN PENDIENTE

No se asigna tiempo de recuperación ni se dibuja una curva sin marcas temporales reales y comparables. Esta sección describe un método pendiente de aplicar.

10. Recomendaciones de mejora

- Distribuir las solicitudes en el tiempo y dejar parte de la cuota de concurrencia libre para la Lambda consumidora y otras funciones.
- Medir y mejorar el arranque de Spring Boot, evaluando la inicialización asíncrona compatible con el adaptador de AWS.
- Revisar cuánto tardan Hibernate/JPA y la conexión a Neon al iniciar, sin eliminar controles de validación o aumentar conexiones sin medir.
- Conservar el uso de SES Simulator y los límites por usuario para evitar envíos reales o costos inesperados.
- Registrar códigos HTTP agrupados, duración y timestamps sin incluir JWT, contraseñas ni contenido de los mensajes.
- Mantener métricas históricas para poder distinguir ejecución en frío, procesamiento normal, mensajes pendientes y recuperación.

11. Conclusiones

La prueba permitió observar que una misma arquitectura puede responder bien a una carga pequeña y, sin embargo, presentar muchos rechazos cuando crece la concurrencia. En la línea base se aceptaron las 30 solicitudes, mientras que en la ráfaga de 50 usuarios fallaron 40. AWS mostró que

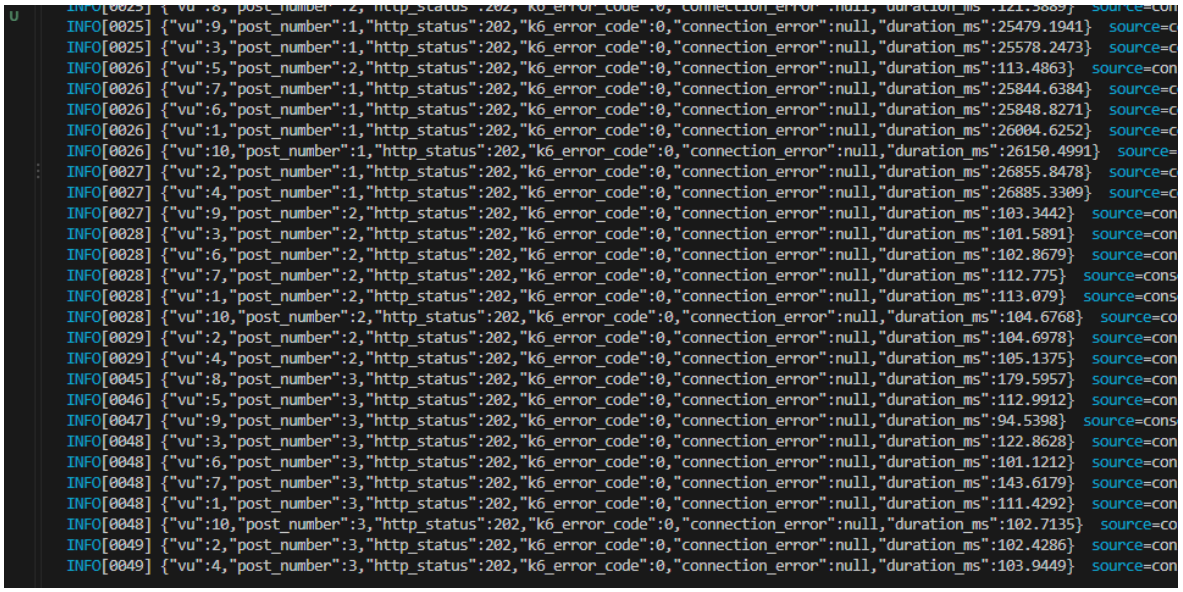
la cuenta estaba limitada a 10 ejecuciones simultáneas, lo que explica de forma consistente la mayor parte del problema observado.

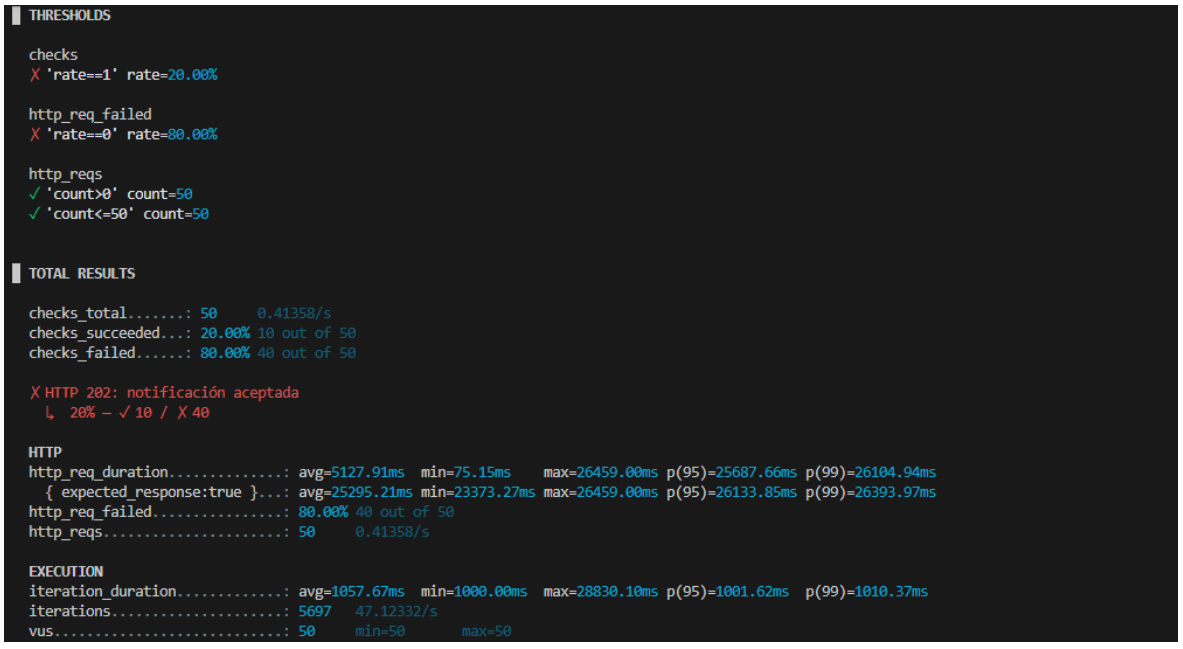
También quedó claro que la lentitud de los primeros arranques merece atención: las solicitudes aceptadas con 50 usuarios tardaron alrededor de 25 segundos en promedio. Como resultado, las mejoras más razonables son controlar la concurrencia y estudiar el inicio del backend antes de aumentar la carga.

La práctica reunió evidencias útiles de k6, API Gateway, Lambda, SNS, SQS y SES. Los escenarios de mayor tamaño y el tiempo real de recuperación quedaron pendientes y así se presentan en este informe.

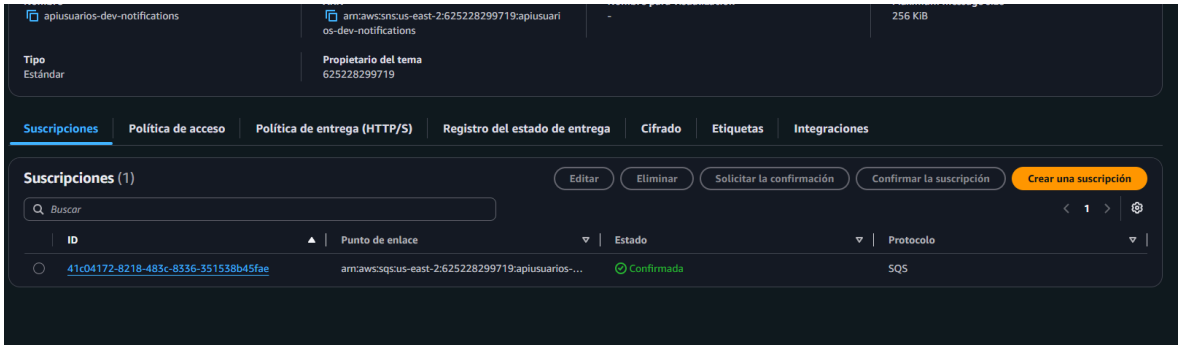
Anexo. Evidencias de la práctica

Las siguientes capturas fueron tomadas durante las pruebas y compartidas para la elaboración del informe. No sustituyen los archivos JSON ni las exportaciones completas de CloudWatch, que se conservan por separado en el proyecto local.











Fuentes consultadas

INFORME-TECNICO.md; RESUMEN-VISUAL.html; ACTIVIDAD4-DEGRADACION.md; ACTIVIDAD5-RECUPERACION.md (archivos del proyecto, 16 de octubre de 2026).

Resúmenes originales k6 referenciados por el informe: actividad1-20261016-005817.json y actividad2-50vus-20261016-014407.json (conservados en el proyecto local).

AWS Lambda — Métricas y monitoreo: https://docs.aws.amazon.com/lambda/latest/dg/monitoring-metrics-types.html

AWS Lambda — Errores de integración con API Gateway: https://docs.aws.amazon.com/lambda/latest/dg/services-apigateway-errors.html

Amazon SQS — Métricas de CloudWatch: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-available-cloudwatch-metrics.html